# 颜色识别与质心

## 从零手搓 Step2 教程

2026 FPGA 创新设计赛道（安路）赛题二

视觉方向 • Step2_color_centroid

读者：零基础队员（建议先读完 Step1 教程）
目标：理解并手搓视觉检测链——从像素流里”认出”目标并算出坐标
前置要求：会 Verilog 基础语法、跑过 ModelSim 仿真
预计学习时间：12～16 天（每天 2～3 小时）

**本教程与你手上的代码是什么关系**

Step1 学会了”把像素接住”；Step2 要做的，是从这些像素里**认出颜色目标**并算出它的**画面坐标**——这是自动跟踪的”眼睛”。

# 目录

**第一部分　从零学习教程** **4**

**1　你在造什么** **4**
1.1　一句话 . . . 4
1.2　整条检测链 . . . 4
1.3　做完是什么效果 . . . 4

**2　阶段 0：图像与颜色基础（1～2 天）** **5**
2.1　像素：三个数描述一个颜色 . . . 5
2.2　RGB 判颜色的麻烦 . . . 5
2.3　HSV：换个说法描述颜色 . . . 5
2.4　练习 . . . 6

**3　阶段 1：定点算术——FPGA 的数学（2～3 天）** **6**
3.1　FPGA 不会做小数 . . . 6
3.2　截断陷阱：» 是” 向下取整” . . . 7
3.3　定点验证的重要性 . . . 7
3.4　练习 . . . 7

**4　阶段 2：RGB→HSV 的电路原理（3～4 天）** **7**
4.1　第 1 步：找最大、最小、差值 . . . 7
4.2　第 2 步：明度 V 和饱和度 S . . . 8
4.3　第 3 步：色相 H——判断在哪个扇区 . . . 8
4.4　第 4 步：扇区内插值 . . . 8
4.5　第 5 步：拼装完整公式 . . . 8
4.6　2 级流水线 . . . 9
4.7　练习 . . . 9

**5　阶段 3：阈值分割（2 天）** **9**
5.1　二值化：每个像素过一遍筛子 . . . 9
5.2　H 窗口环绕——红色跨 0 度的坑 . . . 10
5.3　S/V 门限各管什么 . . . 10
5.4　8 组预置：现场按键切换 . . . 10
5.5　练习 . . . 10

**6　阶段 4：去噪——3×3 多数表决（3～4 天）** **10**
6.1　噪声从哪来 . . . 10
6.2　3×3 窗口 + 多数表决 . . . 11
6.3　边界怎么办：复制 . . . 11
6.4　行缓存：怎么” 看到上一行” . . . 11
6.5　练习 . . . 12

**7 阶段 5：质心与包围框（3 天） 12**
7.1 质心：所有目标像素的” 重心” . . . 12
7.2 为什么用质心，不用包围框中心 . . . 12
7.3 帧累加器的电路结构 . . . 13
7.4 除零保护与最小像素门限 . . . 13
7.5 包围框：4 个极值寄存器 . . . 13
7.6 练习 . . . 13

**8 阶段 6：跑通检测链测试（1～2 天） 14**
8.1 一键回归 . . . 14
8.2 怎么读测试结果 . . . 14

**第二部分 工程实现详解 15**

**9 工程总览 15**
9.1 文件清单 . . . 15
9.2 数据流与延迟账 . . . 15
9.3 三条设计原则 . . . 16

**10 模块详解：rgb2hsv 16**
10.1 为什么用 3840 这个数 . . . 16
10.2 两级流水的分工 . . . 16
10.3 已知特性：品红为什么是 H=0 . . . 17

**11 模块详解：hsv_inv_lut（生成文件） 17**

**12 模块详解：hsv_threshold 17**

**13 模块详解：bin_smooth3x3——从 v3 到 v7 的重构 18**
13.1 为什么是” 多数表决” 而不是均值 . . . 18
13.2 v3 架构：移位延迟线（失败的开始） . . . 18
13.3 推倒重来：v7 索引行缓存（自对齐结构） . . . 18
13.4 行尾窗口：一个容易漏的输出约定 . . . 19
13.5 边界复制语义表（与金标准逐位对应） . . . 19

**14 模块详解：centroid_accum 20**
14.1 帧累加器：三个加数器 + 一组极值 . . . 20
14.2 最阴的 bug：除法快照（迭代 8） . . . 20
14.3 恢复法除法与” 商末位预测” . . . 20
14.4 有效性、丢帧计数与输出 . . . 21

**15 模块详解：detect_top（窃流旁路顶层） 21**
15.1 为什么挂在 96bit 点的 AWB 之后 . . . 21
15.2 纯旁路的写法 . . . 21

15.3 顺手带出的三路输出 . . . . . 21

**16 测试平台设计** **22**
16.1 向量生成器：所有期望值只有一个来源 . . . . . 22
16.2 tb_detect_chain：全尺寸真实场景 . . . . . 22
16.3 三个向量文件格式坑（全部来自实战） . . . . . 22

**17 验证结果** **22**
17.1 回归矩阵 . . . . . 23
17.2 Step2 阶段抓到的真缺陷 . . . . . 23

**18 上板挂载说明（给上板组）** **23**

**19 小结：Step2 的可复用产出** **24**

**结语** **25**

**总结与下一步** **25**
Step2 学到了什么 . . . . . 25
下一步学什么 . . . . . 25

# 从零学习教程

## 1 你在造什么

### 1.1 一句话

**从每秒 5500 万个像素里，实时找出”红色目标”在哪里，并给出坐标。**

上一份教程（Step1）你学会了”把摄像头的数据接住”；这一份要在这条像素流上做三件事：

**Step2 要回答的三个问题**

1. **这个像素是什么颜色？** 把 RGB 换算成”人更容易判定的颜色描述”（HSV）
2. **它是不是我们要的目标色？** 用一个”阈值窗”把画面上属于目标的像素挑出来（掩码）
3. **目标在画面哪里？** 把所有目标像素的坐标平均，得到”重心”（质心）；顺带算包围框

### 1.2 整条检测链

```
96bit 像素流 ──> ser96to24 ──> rgb2hsv ──> hsv_threshold
（Step1 已做）     1 像素/拍      RGB→HSV       挑出目标" 红色" 像素
                                                      |
                                                      ▼
目标坐标 <── centroid_accum <── bin_smooth3x3 <── 掩码（1 像素 1 bit）
（cx，cy）      帧内累加求质心       3×3 多数表决去噪
```

整条链每拍处理 1 个像素，74MHz 时钟下每秒能过 5500 万像素——和摄像头 720p60 的像素率完全匹配，不多不少。

### 1.3 做完是什么效果

屏幕上：红色目标上出现绿色十字准星和红色包围框，左上角状态行显示坐标：

```
TRK0 LOCK X0560Y0320      ← 认出目标（LOCK），坐标 (560, 320)
TRK0 LOST  X----Y----     ← 画面里没有目标（LOST）
```

**学完 Step2 你能做到**

- 解释 RGB 和 HSV 的区别，说清为什么判颜色要用 HSV
- 手算一个像素的 HSV 值和阈值判决结果
- 理解”查表代替除法”“移位代替乘除”这些 FPGA 常用技巧
- 看懂行缓存、3×3 窗口、质心累加器的电路结构
- 跑通检测链的 5 个测试，并知道每个测试在验什么

## 2 阶段 0：图像与颜色基础（1～2 天）

### 2.1 像素：三个数描述一个颜色

屏幕上每个小格子叫一个**像素**（pixel），由三个数字组成：

```
像素 = (R, G, B)     每项 0～255

(255,   0,   0) = 纯红          (  0, 255,   0) = 纯绿
(  0,   0, 255) = 纯蓝          (255, 255, 255) = 白
(128, 128, 128) = 中灰          (  0,   0,   0) = 黑
(224,  32,  32) = 暗一点的红   ← 本项目要跟踪的目标色
```

一个 1280×720 的画面有 92 万个这样的三元组，每秒刷新 60 次。FPGA 要在 16.7 毫秒内把每一帧全部处理完——这就是” 实时” 的含义。

### 2.2 RGB 判颜色的麻烦

想找出” 红色目标”，最直接的想法：

```
if (R > 150) && (G < 80) && (B < 80) then 是红色
```

这个写法在实验室灯光下能工作，但一换环境就崩：

| 场景 | 同一块红布的颜色 | R>150? | 判定结果 |
|---|---|---|---|
| 室内正常光 | (224, 32, 32) | 是 | 正确识别 |
| 室外强光 | (255, 120, 120) | 是 | 勉强正确（G/B 已超标） |
| 阴影下 | (90, 12, 12) | 否 | 漏检！ |
| 白炽灯偏黄 | (240, 110, 60) | 是 | 勉强正确 |

**问题根源：RGB 三个数字里，” 有多亮” 和” 是什么颜色” 是混在一起的。**光照变强时三个数一起变大；光照变弱时一起变小。你想判断” 是不是红色”，却不得不同时应付亮度的干扰。

### 2.3 HSV：换个说法描述颜色

把” 三个混在一起的数” 换成” 三个各管一事的数”：

**HSV 三个分量**

- **H（Hue，色相）**：这是**什么颜色**。0~360 度一圈
- **S（Saturation，饱和度）**：颜色**有多纯**。0 = 灰，越大越鲜艳
- **V（Value，明度）**：颜色**有多亮**。0 = 黑，越大越亮

色相环（H 的取值）：



```
300° 品红 ─┤   ├─ 60° 黄
           │   │
270°      ─┤   ├─ 90°
           │   │
240° 蓝   ─┤   ├─ 120° 绿
           │   │
      210° ─┘   └─ 150°
            180° 青
```

判" 红" 只需：H 在 335°～360° 或 0°～25°（红色在环上是跨 0 度的一段）
光照变化主要影响 S 和 V，对 H 的影响小得多——这就是 HSV 的好处

**一句话记住**

RGB 是" 三盏灯的亮度"；HSV 是" 什么颜色 + 多纯 + 多亮"。要和光照变化对抗，就用后者。

### 2.4 练习

1. 打开 Python，装好 `opencv-python`，运行：

```
import cv2, numpy as np
rgb = np.uint8([[[224, 32, 32]]])        # 一个红色像素（注意 cv2 要 BGR!）
hsv = cv2.cvtColor(np.uint8([[[32, 32, 224]]]), cv2.COLOR_BGR2HSV)
print(hsv)   # 看看输出的 H/S/V 是多少
```

2. 找一张照片，看" 红色物体" 和" 白色墙" 的 HSV 分别是什么样
3. 思考题：白色的 H 和 S 应该是多少？（提示：白色的" 颜色" 是什么？）

## 3 阶段 1：定点算术——FPGA 的数学（2～3 天）

### 3.1 FPGA 不会做小数

FPGA 里没有" 浮点数" 概念（至少本项目不用）。所有数都是整数。那" 除以 delta" 这种运算怎么办？两个办法：**移位**和**查表**。

**技巧 1：移位代替乘除 2 的幂**

```
x << 1   等于   x × 2
x << 2   等于   x × 4
x >> 1   等于   x ÷ 2（向下取整）
x >> 6   等于   x ÷ 64
```

移位在电路里几乎不花资源（就是接线方式变了），硬件里是" 免费" 的。

**技巧 2：查表代替除法**

色相计算需要”除以 delta”（delta 是 1~255 之间某个数）。除法的电路很大很慢——但我们可以在**设计阶段**把 256 个可能的商全算出来，预先放进一张小表（ROM），运行时”查一下”就行：

```
inv[1] = 3840, inv[2] = 1920, inv[3] = 1280, ...
inv[d] = floor(3840 / d)      d = 1～255

表大小：256 项 × 12 bit ≈ 3.8 Kbit（一块小 ROM 就装下）
运行时：查表只要 1 拍，比除法器又快又省
```

### 3.2 截断陷阱：» 是”向下取整”

**负数右移会变小**

```
-7 >> 1  =  -4      （不是 -3！向下取整：-3.5 → -4）
```

本项目的色相计算里有负数（G−B 可能为负），右移的取整方向必须和 Python 金标准模型完全一致，否则 RTL 和模型会差 1，验证不通过。

**规则：**Python 的 `>>` 和 Verilog 的带符号 `>>>` 都是向下取整——两边写法对齐，就能逐位一致。

### 3.3 定点验证的重要性

**本项目的工作方式：金标准模型**

1. 先用 Python 写一个”标准答案”版本（`tools/golden_model.py`）
2. 再让 RTL 实现同样的算法，要求**逐位一致**（不是”差不多”）
3. 用 1 万条随机向量测试，任何一位不同都算失败

这叫”位精确（bit-exact）验证”。因为 FPGA 里没有”约等于”，差 1 就可能让阈值判决翻转，目标时有时无。

### 3.4 练习

1. 手算：`5 >> 1`、`-5 >> 1`、`7 >> 2`、`-7 >> 2`
2. 用 Python 验证你的答案（注意 Python 的负数是无限精度，`-5 >> 1 = -3`——想想为什么和 Verilog 不同）
3. 打开 `tools/golden_model.py`，找到 `build_inv_lut()` 函数，看看倒数表怎么建的

## 4 阶段 2：RGB→HSV 的电路原理（3～4 天）

这一节是整个 Step2 的核心。跟着五步走，你就能手推任意像素的 HSV。

### 4.1 第 1 步：找最大、最小、差值

```
输入：R, G, B（各 8 bit）

max   = max(R, G, B)      三个数里最大的
min   = min(R, G, B)      三个数里最小的
```

```
delta = max - min          最大最小之差（" 颜色的纯不纯"）

例子：(224, 32, 32)  →  max=224, min=32,  delta=192
```

电路实现：三个数两两比较，找最大最小各需 2 次比较（比较器很便宜）。

### 4.2 第 2 步：明度 V 和饱和度 S

```
V = max           有多亮（纯红的 V=224，说明它挺亮）
S = delta         有多纯（纯红 delta=192；灰色三个数相等 → delta=0）

注意：标准 HSV 里 S 要除以 max 归一化（S = delta/max × 255），
本项目为了省掉一次除法，直接用 delta 代替——效果够用（详见 Part II）。
```

### 4.3 第 3 步：色相 H——判断在哪个扇区

看哪个分量最大：

```
R 最大 → 颜色在 0°～60° 附近（红～黄）
G 最大 → 颜色在 120°～180° 附近（绿～青）
B 最大 → 颜色在 240°～300° 附近（蓝～品红）

本项目简化版：三个扇区，基值 0° / 120° / 240°，
    优先级 R > G > B（两个相等时算前者）

例子：(224, 32, 32)：R 最大 → 基值 0°
```

### 4.4 第 4 步：扇区内插值

在扇区里还要细分”偏移多少度”，公式：

```
R 最大时：   h = (G - B) × 3840 / delta / 64      ← 用到查表！
（其他扇区换分子：G 最大用 B-R，B 最大用 R-G）

为什么是 3840？  因为 3840/64 = 60，正好是一个扇区的宽度（60°）
除以 64 用 >>6 移位实现

例子：(224, 32, 32)：G-B = 0  →  h = 0  →  H = 0° + 0 = 0°（就是纯红，对上了）
```

**查表怎么用**

`inv[delta]` 查到 `3840/delta` 的预计算值（12 bit），然后：

```
h = (G-B) × inv[delta] >> 6
```

一次乘法 + 一次移位，没有任何除法——这就是定点技巧的威力。

### 4.5 第 5 步：拼装完整公式

```
H = 扇区基值 + clamp(h, 0, 60)
```

```
clamp 的原因：h 由差值算出，理论上在 0～60 之间，
    但数值误差可能溢出边界，钳一下保险

最终输出：
    H：9 bit（0～359）    S：8 bit（=delta）    V：8 bit（=max）
```

**已知简化的代价（必须知道）**

本项目用"3 扇区"简化，H 只能落在：**0～60°（红黄区）、120～180°（绿青区）、240～300°（蓝紫区）**。中间色（如品红 300°、黄绿 90°）会被并入相邻扇区。

**实际影响：**品红色（R=B>G）会被判成 H=0——和红色混在一起。对策：靠 S/V 门限过滤 + 演示时避免品红干扰物（详见 Part II 的实测场景）。

完整 6 扇区版本要多一次比较和更多逻辑，本项目为省资源选了 3 扇区。

### 4.6 2 级流水线

```
第 1 拍：算 max/min/delta，选好扇区差值，连同 RGB 一起存下来
第 2 拍：查表（拿 delta 查 inv）→ 乘法 → 移位 → clamp → 出 H/S/V

为什么分 2 拍？组合逻辑太长（比较 + 查表 + 乘法叠在一起），
74MHz 时钟下时序过不去；拆成 2 拍每拍都轻松
为什么不拆更多？延迟只为了让电路跑得动，多了没意义
```

### 4.7 练习

1. 手算这 4 个像素的 HSV（用 `inv` 表近似）：`(255,0,0)`、`(0,255,0)`、`(0,0,255)`、`(128,128,128)`
2. 和 Python 的答案对一下（`tools/golden_model.py` 里有 `rgb_to_hsv`）
3. 想想：`(255, 255, 0)`（纯黄）的 H 是多少？为什么这个简化版本给不出 60°？

## 5 阶段 3：阈值分割（2 天）

### 5.1 二值化：每个像素过一遍筛子

有了 HSV，判断"是不是目标色"就是三个条件同时成立：

```
mask = (H 在窗口内) && (S >= s_lo) && (V 在 [v_lo, v_hi] 内)
```

输出 `mask` 是 1 个 bit：1 = 目标像素，0 = 背景。整帧过完得到一张"黑白图"（掩码）。

### 5.2 H 窗口环绕——红色跨 0 度的坑

**红色的窗口是” 两段”**

红色在色相环上跨 0°：335°～360° 和 0°～25° 都算红色。

不能用简单的 `(H >= 335) && (H <= 25)`——这个条件永远不成立（335 > 25）!

**正确写法**：判断 `h_lo > h_hi` 时改用” 或”：

```
in_h = (h_lo > h_hi) ? ((H >= h_lo) || (H <= h_hi))
                     : ((H >= h_lo) && (H <= h_hi));
```

### 5.3 S/V 门限各管什么

| 条件 | 挡住什么 | 例子 |
|---|---|---|
| S ≥ s_lo | 灰、白、黑（不纯的颜色） | 白墙 S=0，被挡掉 |
| V ≥ v_lo | 太暗的（阴影里的噪声） | 暗红阴影 V=40，被挡掉 |
| V ≤ v_hi | （可选）过曝的 | 反光高亮区 |

### 5.4 8 组预置：现场按键切换

```
组 0: 红   H 在 [335,25]  S>=80   V 在 [60,255]     ← 默认，跟踪红色目标
组 1: 绿   H 在 [110,145] S>=60   V 在 [40,255]
组 2: 蓝   H 在 [200,250] S>=60   V 在 [40,255]
组 3: 橙黄 / 组 4: 青 / 组 5: 品红 / 组 6: 白 / 组 7: 黑
```

现场换目标颜色（比如评委拿个绿色球），按一下按键切到组 1 就行——不用改代码、不用重新烧板子。

### 5.5 练习

1. 手算：`(224,32,32)` 在组 0 → mask = ?（H=0, S=192, V=224）
2. 手算：`(128,128,128)` 在组 0 → mask = ?（想想 S）
3. 打开 `hsv_threshold.v`，找到 case 语句里的 8 组参数

## 6 阶段 4：去噪——3×3 多数表决（3～4 天）

### 6.1 噪声从哪来

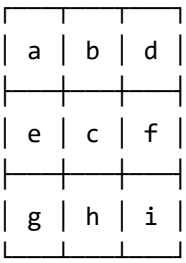
掩码里会混进” 假目标”：传感器噪点、边缘反光、压缩伪影……它们通常是零散的孤立点。看这张示意（1 = 目标像素）：

```
有噪声的掩码                  去噪后
0 0 0 0 0 0 0                 0 0 0 0 0 0 0
0 1 1 1 0 0 0                 0 1 1 1 0 0 0
0 1 1 1 0 1 0   ← 噪点        0 1 1 1 0 0 0
0 1 1 1 0 0 0                 0 1 1 1 0 0 0
0 0 0 0 0 0 0                 0 0 0 0 0 0 0
```

### 6.2 3×3 窗口 + 多数表决

对每个像素，看它和周围 8 个邻居（共 9 个），数一数有几个是 1：

邻居窗口（以中心像素 c 为核心）



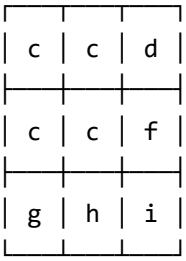

9 个格子里 1 的个数 >= 5 → 输出 1
否则 → 输出 0

**为什么是” 多数表决” 而不是” 平均数”？** 9 个里有 5 个以上是 1 才保留——孤立噪点（周围都是 0）被干掉，真实目标（成片都是 1）被保留。这就像投票：群众的眼睛是雪亮的。

### 6.3 边界怎么办：复制

画面最外圈像素没有完整邻居。本项目的策略：**复制边界**（画面外的像素按” 最近的画面内像素” 算）。

左上角 (0,0) 的实际窗口（复制边界后）： 画面外的位置一律复制最近像素




这样 (0,0) 也能得到一个完整的 9 格窗口

### 6.4 行缓存：怎么” 看到上一行”

像素是一行一行流过来的。要组成 3×3 窗口，需要同时看到” 上一行” 和” 上上一行”——但那些像素已经流走了。怎么办？

**行缓存（line buffer）**

用一个能存**一整行**的存储器，把流过去的像素留下来：

- 当前行：正在流的像素
- 行缓存 1：存上一行（1280 个 bit）
- 行缓存 2：存上上一行

三个来源凑齐，3×3 窗口就齐了。

**！本项目最难的坑（Part II 有完整复盘）**

”怎么对齐三行”看起来简单，实际连踩 5 个坑（FIFO 收支差、移位方向、窗口取列差一拍、延迟线比设计短一格……）才做对。

最终方案：**索引行缓存**——读写用同一个列指针，”读到上一行的同一列”从结构上就成立，不需要手工推相位。

初学建议：先接受”最终版是对的”去看 Part II 的复盘，比试图自己从零推快得多。

### 6.5 练习

1. 给这张小掩码（5×5）手工做一遍 3×3 多数表决（边界复制）：

```
0 0 0 0 0
0 1 1 1 0
0 1 0 1 0      ← 中心有个洞，表决后会被填上吗？
0 1 1 1 0
0 0 0 0 0
```

2. 用 Python 对同一张图跑 `golden_model.smooth3x3_majority`，对比你的答案

## 7 阶段 5：质心与包围框（3 天）

### 7.1 质心：所有目标像素的”重心”

目标是一块由很多像素组成的区域。用哪一点代表它的位置？——**质心**（所有目标像素坐标的平均值）：

```
假如目标是这 3 个像素：(x, y) = (10, 4), (11, 4), (12, 4)

cx = (10 + 11 + 12) / 3 = 11        ← x 的平均
cy = ( 4 +  4 +  4) / 3 = 4         ← y 的平均
目标位置 = (11, 4)
```

一般公式（m 为掩码：1 = 目标像素）：

```
对全画面的每个像素：
    count = Σ m                  目标像素总数
    sum_x = Σ (x × m)            "x 的和", m=1 时才累加
    sum_y = Σ (y × m)

帧末（一帧过完）算一次：
    cx = sum_x / count
    cy = sum_y / count
```

### 7.2 为什么用质心，不用包围框中心

| 方案 | 优点 | 缺点 |
|---|---|---|
| 包围框中心 | 简单（4 个寄存器） | 目标形状不规则时偏离视觉中心 |
| **质心（选定）** | 反映”质量分布”，更符合直觉 | 需要一次除法 |

本项目两个都算：质心用来画准星和控云台，包围框用来画红框。

### 7.3 帧累加器的电路结构

```
每个像素到来时（1 拍内完成）：
    if (mask) begin
        count <= count + 1;          // 21 bit 计数器
        sum_x <= sum_x + x;          // 32 bit 累加器
        sum_y <= sum_y + y;
    end

  x 计数器：每拍 +1，行尾清零（0～1279）
  y 计数器：每行 +1，帧首清零（0～719）

帧末（第 719 行的行尾）：
    启动除法状态机：sum_x / count → cx
                    sum_y / count → cy
    除法大约 70 拍完成——一帧的消隐期有 7 万多拍，绰绰有余
```

**为什么除法要放到帧末？**

除法器是”大电路”，如果每个像素都做除法（每拍一次），要么资源爆炸要么降频。

而质心只需要**每帧一个结果**——把除法安排在帧末的消隐期（画面不出像素的那段空档），慢慢地算，完全不影响像素流。

### 7.4 除零保护与最小像素门限

**两个必须处理的边界情况**

1. **count = 0**（画面里没有目标）：除法除零会得到垃圾值——必须判断”count 为 0 就直接输出无效”，不做除法
2. **count 很小**（比如 5 个杂点）：质心会被噪点带偏——设一个最小门限（本项目 30 个像素），不够就当没目标

### 7.5 包围框：4 个极值寄存器

```
if (mask) begin
    if (x < min_x) min_x <= x;       // 最左
    if (x > max_x) max_x <= x;       // 最右
    if (y < min_y) min_y <= y;       // 最上
    if (y > max_y) max_y <= y;       // 最下
end
  （另外需要一个 seen 标志：帧内第一次见到目标时把四个值都设成当前坐标）
```

### 7.6 练习

1. 手工算：三个目标像素 (10,4), (11,4), (12,4) 的质心和包围框
2. 思考：如果目标是 L 形（缺一角），质心会在图形的内部还是外部？
3. 读 `centroid_accum.v` 的除法状态机部分，理解”恢复法除法”

# 8 阶段 6：跑通检测链测试（1～2 天）

## 8.1 一键回归

```
cd D:\fpga-ptz-tracker\tools
python run_all.py
```

检测链相关的 5 个测试：

| 测试 | 验证什么 | 规模 |
|---|---|---|
| tb_rgb2hsv | RGB→HSV 位精确 | 10010 条随机 + 角点向量 |
| tb_hsv_threshold | 阈值判决位精确 | 4000 条 × 8 组预置 |
| tb_bin_smooth | 3×3 平滑位精确 | 12288 像素 |
| tb_centroid | 质心/包围框/除零 | 3 帧迷你几何 |
| tb_detect_chain | 全链端到端 | 3 帧真实场景（含干扰物） |

## 8.2 怎么读测试结果

```
--- tb_rgb2hsv ...
    ==> PASS                          ← 10010 条向量全部逐位一致

--- tb_detect_chain ...
    # FRAME0 OK v=1 cnt=8096 cx=604 cy=345
    # FRAME1 OK v=1 cnt=32592 cx=644 cy=391
    # FRAME2 OK v=1 cnt=221 cx=107 cy=108
    ==> PASS                          ← 3 帧场景全部与金标准一致
```

每个 OK 行显示的是检测结果：有效性、像素数、质心坐标。和 chain_exp.txt（Python 金标准算的期望值）逐字段对比。

**Step2 学习清单（全部打勾就过关）**

- 能解释 RGB 和 HSV 的区别，说清 S/V 门限各挡什么
- 能手算一个像素的 HSV 和阈值判决
- 能解释” 查表代替除法” 和” 移位代替乘除”
- 能画出行缓存 + 3×3 窗口的框图
- 能解释质心公式和除零保护
- 跑通 5 个测试，能说清每个测试在验什么

# 工程实现详解

**这一部分写什么**

上面的 Part I 是” 从零学知识”；这一部分换一个角度：把仓库里**真实做出来的 Step2 工程**逐模块讲透——每个文件为什么这么写、每一步踩过什么坑、怎么用金标准证明它是对的。学完 Part I 再读这里，你会看到” 教科书知识” 是怎样变成仓库里 916 行可综合 RTL 的。

对应目录：`03_vision/step2_color_centroid/`

对应代码：`03_project/rtl/` 下 6 个 RTL 文件（916 行）

对应证据：`03_project/sim/` 下的向量与期望文件、`04_ 迭代文件/迭代记录 _v1 到 v7.md`

## 9 工程总览

### 9.1 文件清单

| 文件 | 类型 | 行数 | 职责 |
|---|---|---|---|
| `rgb2hsv.v` | RTL | 100 | RGB888 到 HSV 转换，2 级流水 |
| `hsv_inv_lut.v` | 生成 | 267 | 倒数表 ROM，脚本生成禁止手改 |
| `hsv_threshold.v` | RTL | 58 | 8 组预置阈值二值化 |
| `bin_smooth3x3.v` | RTL | 120 | 3x3 多数表决去噪（v7 索引行缓存） |
| `centroid_accum.v` | RTL | 208 | 帧累加求质心/包围框 + 帧末除法 |
| `detect_top.v` | RTL | 163 | 检测链顶层（96bit 窃流旁路） |
| `tb/tb_*.v` | 测试 | 5 个 | 单模块 4 个 + 整链 1 个 |
| `sim/*.hex/.txt` | 向量 | — | 金标准生成的激励与期望 |

### 9.2 数据流与延迟账

```
96bit AXI-S (4px/beat, AWB 后)
   |  ser96to24 (Step1 模块, FIFO 降带宽)
   v
1 像素/拍 (24bit)
   |  rgb2hsv          延迟 2 拍
   v
H/S/V (9+8+8 bit)
   |  hsv_threshold    延迟 1 拍
   v
mask (1bit)
   |  bin_smooth3x3    延迟 ~3 拍 + 2 行行缓存
   v
mask_smooth
   |  centroid_accum   帧内累加, 帧末 ~66 拍除法 (消隐期)
   v
```

```
目标包（坐标/包围框/计数）-> 云台 / OSD
```

整链延迟约 9 拍（不含帧累加），全部工作在 1 像素/拍 @ 74.25 MHz。**关键约束：**除法必须在帧消隐期（约 1440 拍）内完成，66 拍绰绰有余，所以选了最省资源的串行恢复法除法，而不是并行除法器。

### 9.3 三条设计原则

**本工程反复验证过的三条原则**

**1. 金标准先行：**所有算术先在 `tools/golden_model.py` 用 Python 定点实现，RTL 逐位复刻。失配时先对照 Python 找差异，再改 RTL。

**2. 自对齐结构优先：**宁可多用一点逻辑资源，也要选”结构上不可能错位”的方案（详见 bin_smooth3x3 的 v3 到 v7 重构）。

**3. 流水线标志全程随行：**valid/sof/eol 三级标志跟着数据一起打拍，任何模块都不许”自己数拍子猜边界”。

## 10 模块详解：rgb2hsv

### 10.1 为什么用 3840 这个数

HSV 标准公式里有个除以 delta 的运算，FPGA 不做除法。做法是把 1/delta 提前算成表：`inv[d] = floor(3840/d)`。3840 的来历：

$$3840 = 60 \times 64$$

- 乘 60：让扇区内插值直接出”度”（每扇区 60 度）；- 乘 64：最后用 `>>6` 除掉它，移位代替除法。于是 `h = (diff × inv[delta]) >> 6` 的值域恰好是 0..60，加扇区基值（0/120/240）就是完整色相。

### 10.2 两级流水的分工

```
第 0 拍（组合 + 打拍）：
  max/min 比较树，delta = max - min
  扇区判定：is_r = (R>=G && R>=B);  is_g = !is_r && (G>=R && G>=B)
  扇区差值：R 扇区取 (G-B)，G 扇区取 (B-R)，B 扇区取 (R-G)   [9bit 有符号]

第 1 拍（查表 + 乘 + 移位）：
  inv = inv_lut[delta]                       // 256x12 ROM
  h   = clamp((diff * inv) >>> 6, 0, 60)   // 有符号乘 9x13 -> 22bit
  H   = 扇区基值 + h                         // delta==0 时强制 H=0
```

为什么差值必须 9 bit 有符号？因为 G-B 可以是负数（比如纯红 G=B=0 没问题，但 (0,255,0) 的 B-R = -255）。RTL 里写成：

```
wire signed [8:0] diff_r;
always @(*) begin
    if (is_r)      diff_r = $signed({1'b0, g}) - $signed({1'b0, b}); // G-B
```

```
    else if (is_g) diff_r = $signed({1'b0, b}) - $signed({1'b0, r}); // B-R
    else           diff_r = $signed({1'b0, r}) - $signed({1'b0, g}); // R-G
end
```

**位精确的三个要点（每次改这个模块都要默念）**

**1.** Python 的 `>>` 对负数就是向下取整，Verilog 必须用 `>>>`（算术右移）配合 `signed` 才能一致；用 `>>` 或漏掉 signed 会让负半区（纯红附近）差 1 度。

**2.** `4'sd0` 这类字面量如果不显式 signed，混合运算会整体变无符号，差值直接回绕成大正数。

**3.** 金标准里 `h_r = clip(((g-b) * inv) >> 6, 0, 60)` 用的是 numpy 的 floor 除法；RTL 的 `h_raw[6:0]` 截位前先 clamp 到 0，两边对负数语义必须逐位对齐——用 tb_rgb2hsv 的 10010 个向量兜底。

### 10.3 已知特性：品红为什么是 H=0

本实现把 B 最大一律归到 240+ 扇区，且内插差值取 R-G。对标准品红 (255, 0, 255)：R-G = 255-0 = 255，inv[255]=15，h = (255x15)»6 = 59，H = 240+59 = 299——看起来对。但对 (128, 0, 128) 这类”暗品红”：R-G = 128，inv[128]=30，h = (128x30)»6 = 60，H = 300，而标准公式给 300 附近，也在窗内。

真正会翻车的是”红和品红同时出现”的场景：品红 (255,0,255) 的 R 与 B 相等，扇区判定 `is_r = (R>=G && R>=B)` 会把它判成**红色扇区**（R 优先），差值 G-B = -255，h = (-255x15)»6 = -60，clamp 到 0，于是品红拿到 H=0，也就是”和纯红一模一样”。

**这是特性不是 bug**

工程测下来：红色目标检测不受影响（H=0 本来就在红色窗口内），测试场景里放的品红干扰物会被”红窗 + S/V 门限”部分吸收或部分误判——方案对比文档里记录了取舍结论：对比赛演示够用，若后续需要严格区分红与品红，需要扩到 6 扇区全内插（已有演进方案，未上马）。详见 `01_ 方案对比/`——那里有完整的公式推导与实测数据。

## 11 模块详解：hsv_inv_lut（生成文件）

这个 ROM 不允许手写：由 `tools/gen_vectors.py` 生成，头注释明确”自动生成、禁止手改”。生成逻辑就一行：

```
inv[d] = 3840 // d      // d = 1..255, 用整除（floor）
```

为什么不用 `$readmemh` 初始化 ROM？因为要过综合——生成器直接吐出一个 256 项的 `case` 表 RTL，综合器认它是常量，会映射成 Block ROM。Python 金标准用 `build_inv_lut()` 建同一个表，两边同源，差异只可能来自手改——所以禁止手改。

## 12 模块详解：hsv_threshold

58 行的极简模块，但有两个设计点值得学：

```
// H 窗口环绕：h_lo > h_hi 时按" 或" 判断（红色 335..25）
assign in_h = (h_lo <= h_hi) ? (I_h >= h_lo) && (I_h <= h_hi)
```

```
                              : (I_h >= h_lo) || (I_h <= h_hi);
assign mask_c = in_h && (I_s >= s_lo) && (I_v >= v_lo) && (I_v <= v_hi);
```

**1. 8 组预置的 case ROM：**`I_preset_idx` 由按键/拨码切换，0 红 1 绿 2 蓝 3 橙黄 4 青 5 品红 6 白 7 黑。参数表与金标准 `HSV_PRESETS` 逐项同值（评审时可以直接对表）。
**2. 为什么阈值不归一化：**S 用 delta（未除以 max）、V 用 max，都是 8 bit 天然值。这样省掉一次除法，且实拍标定下来门限直观（”红色够不够纯看 S>=80”）。桌面上评测用彩色球/色卡标定过一次参数，写死在预置表里。

## 13 模块详解：bin_smooth3x3——从 v3 到 v7 的重构

这是 Step2 成本最高的一段，也是全工程最有价值的复盘。建议对照 **`04_ 迭代文件/迭代记录 _v1 到 v7.md`** 一起读。

### 13.1 为什么是”多数表决”而不是均值

均值滤波会把 1 像素宽的小目标糊成灰（掩码只有 0/1）；3x3 多数表决（9 格中 >=5 个 1 就输出 1）对孤立噪点（1 个 1 周围 8 个 0）和孔洞（8 个 1 中 1 个 0）都能正确修复，且对目标边缘保持锐利。测试向量里专门造了”椒盐噪声 + 小方块”的帧来验证这一点。

### 13.2 v3 架构：移位延迟线（失败的开始）

第一版按”教科书”写法：

```
// v3 思路（已废弃）
row_delay: 每行一个 1280 位移位寄存器，输出接下一级
z1 <= in; z2 <= z1;                  // 水平 2 级抽头
win = {z2, z1, in} 三列各取三行版本相加
```

想得很美：数据流进移位线，右端自然”看到”过去 2 个像素、过去 2 行。实测三连败：

**v3 的三轮失败（迭代 5、6）**

**失败 1（首错 px=13 行 0）：**移位线读端取错——链最右端是最新数据，代码却按最老读，窗口三列取成 `{k-3, k-2, k-1}`，整体差 2 拍。修复：读端改 bit0，窗口改 `{t1, t0, cur}`。
**失败 2（行 1 起全错）：**垂直延迟线”深度 W”实际只延迟了 W-1 拍（输入端先打一拍才进线），差 1 拍。修复：线加深到 W+1。
**失败 3（行 5 起每行棘轮 +1 错位）：**虚槽 + 预取架构的推入/弹出收支不平衡——每行推入 65 个但只读出 64 个，计数器每行 +1 环绕吃数据。**这是结构性缺陷，补丁打不下去。**

### 13.3 推倒重来：v7 索引行缓存（自对齐结构）

复盘结论写进了迭代记录：”自对齐结构优先于任何需要手工推演相位的紧凑方案。省下的调试时间是数十倍逻辑资源的代价也值得。”
v7 的核心思想一句话：**读指针 = 写指针 = 当前列 x**。

```
(* ram_style = "distributed" *) reg line0_arr [0:W-1];   // row y-1
(* ram_style = "distributed" *) reg line1_arr [0:W-1];   // row y-2
```

```
wire d0 = line0_arr[x];          // 先读：当前列的上两行旧值
wire d1 = line1_arr[x];
always @(posedge I_clk) begin    // 后写：当前像素进 line0，挤出者进 line1
    if (I_valid) begin
        line1_arr[x] <= d0;
        line0_arr[x] <= I_mask;
    end
end
```

同一个 x 下，读到的 d0 就是” 上一行同列”，d1 就是” 上上行同列”——垂直方向不需要任何相位推导，因为读写指针天然一致。再配 2 拍水平 FF 抽头（x-2, x-1）+ 当拍 x，3x3 窗口齐了。这个结构直接抄自官方例程 edge_detect_96（第 9 课官方代码），” 官方例程当参考” 是本工程的一个方法论。

### 13.4 行尾窗口：一个容易漏的输出约定

每行最后一个像素（中心 W-1）的 3x3 窗口需要 W-2, W-1, W-1 复制 三列——但此时流已经走到下一行行首。v7 的处理：把整行的 9 个抽头寄存器一直留着，在下一行 x==0 拍用” 行尾窗口” 公式把上一行最后一个输出补出来：

```
wire [3:0] win_last = {3'd0, t1} + {3'd0, t0} + {3'd0, t0} +
                      {3'd0, m1} + {3'd0, m0} + {3'd0, m0} +
                      {3'd0, b1} + {3'd0, b0} + {3'd0, b0};
wire last_en = (x == 11'd0) && started;
// O_eol <= I_valid && last_en;   // 行尾窗口即每行最后一个输出
```

于是输出流保证每行恰好 1280 个像素、valid 连续、eol 对齐最后一个输出——下游 centroid_accum 直接按流数格点即可，不用知道任何内部相位。**这就是” 输出约定比内部结构重要” 的实例。**

### 13.5 边界复制语义表（与金标准逐位对应）

| 位置 | 规则 | 金标准对应 |
|---|---|---|
| 列 -1 | 复制列 0 | `ext[0] = row[0]` |
| 列 W | 复制列 W-1 | `ext[w+1] = row[w-1]` |
| 行 -1, -2（y=0） | 三行都取当前行 | `top = mid = cur` |
| 行 -1, -2（y=1） | 顶中两行取 row0 | `top = mid = row_mid` |

RTL 里对应的三行选择：y==0 时 t/m 都取当拍输入；y==1 时 t/m 都取 d0（场景 row0 被行缓存挤出后仍在 line1 位置）。

**v7 验收**

tb_bin_smooth：4 帧 64x48（12288 个像素）与金标准 smooth3x3_majority 逐位一致，**12288/12288 全过**。从此没有再出过平滑相关缺陷。

# 14 模块详解：centroid_accum

## 14.1 帧累加器：三个加数器 + 一组极值

```
wire hit = I_valid && I_mask;
// 帧首 sof 清零；命中像素：sum_x += x; sum_y += y; cnt += 1
// bbox：seen 标志法——第一个命中像素初始化 min/max，之后逐像素比较
```

位宽账：sum_x 最大 1280x921600 会溢出 32 位吗？不会——sum_x 的实际上界是 count x 1279 <= 921600 x 1279 ≈ 1.18e9 < 2^31，32 位有符号都够，取无符号更宽裕。cnt 21 位（921600 < 2^20 其实 20 位够，留 1 位余量）。

## 14.2 最阴的 bug：除法快照（迭代 8）

第一版：帧末脉冲启动除法状态机，直接读 sum_x/cnt 边算边用。tb_centroid 显示帧 0/1 结果错、帧 2 对。根因是**时序竞争**：

```
帧 N 的最后一个像素
  -> 除法状态机启动（需要 ~66 拍）
  -> 但下一帧的 sof 清零只要 ~9 拍就到达（流水线延迟短）
  -> 除法算到一半，sum_x/cnt 已被下一帧清零 -> 商是垃圾
```

为什么帧 2 反而”对”？——因为清零点落在这帧除法算完之后的拍上，纯属巧合。
修复方案：**tick 时刻全量快照**。

```
S_IDLE: if (tick_r) begin
    snap_sumx <= sum_x; snap_sumy <= sum_y; snap_cnt <= cnt;   // 定格
    snap_minx <= bmin_x; ... snap_maxy <= bmax_y;
    if (cnt < MIN_PIXELS) st <= S_LATCH;      // 无目标，直接锁存无效
    else begin dividend <= sum_x; ... st <= S_DIV_X; end
end
```

除法与结果锁存**只读 snap__* 寄存器**，与活动累加器完全解耦。这个改动之后，tb_centroid 三帧全过。教训一句话：**跨帧/跨段使用一个累加结果前，先把它”定格”下来**。

## 14.3 恢复法除法与”商末位预测”

除法用最经典的逐位恢复法（shift-subtract），32 位商 32 拍：

```
wire [53:0] rem_sh  = {remainder, dividend[31]};   // 余数左移一位，拉进被除数最高位
wire [21:0] rem_sub = rem_sh[21:0] - divisor;
wire        rem_ge  = (rem_sh[21:0] >= divisor);
// 够减：余数换差，商补 1；不够减：余数左移结果保留，商补 0
quotient <= {quotient[30:0], rem_ge};
dividend <= {dividend[30:0], 1'b0};
```

一个小细节：最后一个商的 bit 是在 bit_cnt==1 那一拍才算出来的，状态机下一拍就走 S_LATCH，来不及用 quotient。所以**商末位与最后一次移位同拍预测锁存**：

```
if (st == S_DIV_X && bit_cnt == 6'd1)
    q_x <= {quotient[30:0], rem_ge};      // 提前拼上最后一位
```

X/Y 两轴串行复用同一个除法器（各 32 拍，共 64 拍），面积省一半。除零保护不用专门写：`cnt < MIN_PIXELS(30)` 时根本不启动除法，直接走 S_LATCH 锁存无效结果——**最小像素门限同时兼任了除零守卫**。

### 14.4 有效性、丢帧计数与输出

```
if (snap_cnt >= MIN_PIXELS) begin
    O_target_valid <= 1'b1; O_cx <= q_x[10:0]; O_cy <= q_y[10:0];
    O_count <= snap_cnt;  O_lost_cnt <= 16'd0;
end else begin
    O_target_valid <= 1'b0; O_cx <= 0; O_cy <= 0;
    if (O_lost_cnt != 16'hFFFF) O_lost_cnt <= O_lost_cnt + 16'd1;  // 饱和
end
```

`lost_cnt` 饱和计数供云台方向做” 连续丢目标” 策略；结果在 `S_LATCH` 拍通过 `O_results_upd` 发出一个脉冲——这个脉冲同时被用做跨域握手（Step4 的 68bit 目标包 CDC load 信号）。

## 15 模块详解：detect_top（窃流旁路顶层）

### 15.1 为什么挂在 96bit 点的 AWB 之后

官方 IPC 链路里，96bit AXI-Stream 的第一站是 ISP 流水线：曝光、白平衡、去噪、转换。挂在 **AWB（自动白平衡）之后、data128_96 之前**，意味着我们拿到的是” 颜色已经校正过” 的像素——红色球在偏黄灯光下也能稳定落在红窗里。如果挂在 AWB 之前，阈值就得跟着灯光漂。

### 15.2 纯旁路的写法

```
assign O_tready = 1'b1;
assign O_tdata  = I_tdata;      // 逐拍零延迟直通
assign O_tvalid = I_tvalid;
assign O_tuser  = I_tuser;
assign O_tlast  = I_tlast;
```

检测链从 `I_tdata` 并行分支出去，**不反压、不插拍、不修改一比特视频**。tb_detect_chain 里专门有一个” 直通一致性” 检查：整个 1382400 拍（3 帧 x 720 x 320 beat）逐拍比对输出与输入，任何一拍的 tdata/tvalid/tuser/tlast 不同就报 PASSTHRU_MISM。
这个结构让” 上板挂载” 变成一句话的事：把视频线接进来、把线接出去，检测结果从侧口出来——不用懂官方 ISP 内部任何细节。

### 15.3 顺手带出的三路输出

- `O_s_*`：串化后的 1ppc 原始流（供 Step4 OSD 当底图）；
- `O_fid`：8 位采集帧号，每个输入帧 sof 自增（进阶（3）延迟可视化）；
- `O_dbg_mask`：平滑后掩码流，仿真里可直接抓波形看” 机器眼里的目标”。

# 16 测试平台设计

## 16.1 向量生成器：所有期望值只有一个来源

`tools/gen_vectors.py` 统一生成 5 组 TB 的激励与期望：

| TB | 向量 | 规模 | 构造方法 |
|---|---|---|---|
| tb_rgb2hsv | rgb2hsv_in/exp | 10010 | 10 个角点 + 10000 随机 RGB |
| tb_hsv_threshold | threshold_in/exp | 4000 | 随机 H/S/V，8 组轮转 |
| tb_bin_smooth | smooth_in/exp | 12288 | 4 帧 64x48 噪声 + 移动方块 |
| tb_centroid | centroid_in/exp | 27648 | 3 帧 128x72 派生几何 |
| tb_detect_chain | chain_beats/flags/exp | 691200 | 3 帧全尺寸真实场景 |

角点必须进向量集：`(0,0,0)`、`(255,255,255)`、纯三原色、纯黄青品红、灰、暗红 `(224,32,32)`——里面藏着 delta=0、差值为负、clamp 触发三个边界，随机向量撞不上这些。

## 16.2 tb_detect_chain：全尺寸真实场景

三帧 1280x720 是” 整链验收”：

| 帧 | 场景 | 金标准结果 | 考察点 |
|---|---|---|---|
| 0 | 红方块 (560,300) 90x90 + 椒盐 | count=8096, cx=604, cy=345 | 噪声不破坏质心 |
| 1 | 红块 + 品红干扰物 + 噪声 | count=32592, bbox 横跨 300..939 | 干扰物部分吸收 |
| 2 | 小目标 15x15 (100,101) | count=221, cx=107, cy=108 | 小目标仍过门限 |

注意第 0 帧：方块 90x90 = 8100 像素，平滑后 8096——边缘 4 个像素被多数表决判成背景，这个 -4 是**金标准算出来的同一个数**，不是近似。位精确验证就是” 连误差都要对得上”。

## 16.3 三个向量文件格式坑（全部来自实战）

**生成器与 TB 的对齐协议**

**1. hex 字段宽度 = 向量位宽 / 4，不许溢出：**`chain_flags.hex` 最初每行写 2 个十六进制字符，但 tuser/tlast 合并只有 2 bit，`$readmemh` 直接按位宽截断，两个标志全毁。改成单字符（值 0..3）。

**2. 期望文件用 $fscanf 读，字段宽度必须自洽：**`chain_exp.txt` 最初 y 坐标用 `%02X` 写，y=300 写出”12C” 三个字符，`$fscanf` 按固定宽度切字段全部错位。规范：坐标类字段一律 `%03X`。

**3. 生成器坐标不许越界：**质心向量生成器最初把方块写到画幅外，numpy 静默裁剪后金标准少算了一块像素，TB” 错得有理说不清”。生成器末尾必须断言所有几何在画幅内。

# 17 验证结果

### 17.1 回归矩阵

| 测试 | 比对粒度 | 向量数 | 结果 |
|---|---|---|---|
| tb_rgb2hsv | H/S/V 三字段逐位 | 10010 | PASS |
| tb_hsv_threshold | mask 逐位 | 4000 | PASS（首过） |
| tb_bin_smooth | mask 逐位 + 行对齐 | 12288 | PASS（v7 重构后） |
| tb_centroid | 8 字段逐字段 | 3 帧 | PASS（快照修复后） |
| tb_detect_chain | 帧结果 + 直通逐拍 | 3 帧全尺寸 | PASS |

`rgb2hsv` 与 `hsv_threshold` 是**一次通过**（首过）的模块——因为它们的算术在 Python 里已经调通，RTL 是抄写而不是设计。这正是” 金标准先行” 的价值：Step2 全部真缺陷都出在**结构**（平滑相位、除法竞争）而不是算术上。

### 17.2 Step2 阶段抓到的真缺陷

| 编号 | 现象 | 根因 | 类型 |
|---|---|---|---|
| S1 | 平滑行 0 首错 | 移位线读端相位取反 | RTL 结构 |
| S2 | 平滑行 1 起错 | 延迟线实际深度 W-1 | RTL 结构 |
| S3 | 每行棘轮 +1 | 虚槽架构收支不平衡 | RTL 架构 |
| S4 | 质心帧 0/1 错 | 除法与下帧清零竞争 | RTL 时序 |
| S5 | 整链编译错 | sim_video_src 缺 timescale | TB |
| S6 | 标志全毁 | flags hex 超位宽截断 | TB 脚本 |
| S7 | 字段错位 | 期望文件字段宽度溢出 | TB 脚本 |

S3 是最贵的一课（v3 到 v6 三个补丁都没救活），最后靠 v7 自对齐结构一次性解决；S4 教会” 结果先定格”。两个教训都写进了迭代记录供后面 Step3/Step4 复用。

**Step2 验收口径**

1）5 个 TB 全绿；2）整链 921600 像素直通逐拍一致；3）三帧真实场景结果与 Python 金标准逐字段一致（含 count 的每个整数）。达到这三条，Step2 视作” 无硬件条件下的位精确完成”。

## 18 上板挂载说明（给上板组）

| 项目 | 内容 |
|---|---|
| 挂载点 | 官方 isp_top 输出 96bit AXI-Stream（AWB 后、data128_96 前） |
| 本例顶层 | `detect_top`（纯旁路，O_tdata 等直接回接官方链路） |
| 时钟 | 与视频像素时钟同域（74.25 MHz），无需 CDC |
| 复位 | 与官方链路同复位（低有效） |
| 按键 | `I_preset_idx[2:0]` 接 KEY 或拨码（换目标颜色） |
| 输出 | 68bit 目标包 + results_upd 脉冲（送 Step3 云台 / Step4 OSD） |
| 资源估计 | 行缓存 2x1280 bit（dis-RAM）+ 256x12 ROM + 1 个 9x13 乘法 + 32bit 除法器——在 PH1P35 上占比很小（TD 综合报告见 Step5） |

上板三步：

1. 在 TD 工程里把 6 个 RTL 文件加入源列表（注意 `hsv_inv_lut.v` 是生成文件，别手改）；
2. 实例化 `detect_top`，把官方 96bit 流接 I_*、输出接回；
3. 用预定按键做 `I_preset_idx`，输出挂 ILA/串口观测（选做）。

**和 Step1 的分工**

Step1（视频链路打通）已经把串化器、CDC、测试图案源验证过了；Step2 的 detect_top 里直接实例化 Step1 的 ser96to24。两个阶段在仓库里是独立目录、独立 TB，但上板时合成一个顶层——接口文档（`06_interfaces`）是它们之间的合同。

## 19 小结：Step2 的可复用产出

- **6 个可综合 RTL 模块**（916 行），全部通过位精确验证；
- **一份金标准 `golden_model.py`**（被 Step3/4/5 继续复用）；
- **5 组向量 + 5 个 TB**，可以随时一键回归；
- **一条方法论**：金标准先行、自对齐结构、结果先定格、标志随行——后面的 Step3/4/5 全部照此执行；
- **一份踩坑档案（`04_` 迭代文件）**，新队员接手时先读它，能省下至少一周的重复调试。

**下一步（Step3 预告）**

Step2 的输出是” 目标坐标”；Step3 要把它变成云台的**运动**：误差计算、Q8.8 定点比例控制、舵机 PWM——以及一次真实的振荡事故（Kp=90 回路增益 1.3 导致跑飞 109 度）的完整复盘。

# 结语

## 总结与下一步

### Step2 学到了什么

- 颜色空间变换：RGB 判颜色为什么不稳，HSV 怎么解决
- 定点算术：查表代替除法、移位代替乘除、floor 与 signed 的坑
- 阈值分割：H 窗口环绕（红色跨 0 度）与 S/V 门限的配合
- 3x3 多数表决去噪：行缓存 + 窗口 + 边界复制
- 质心/包围框：帧累加、快照除法、最小像素门限
- 位精确验证方法学：金标准先行、向量生成、逐位比对

### 下一步学什么

- **Step3 云台闭环**：把坐标变成运动
  - 误差计算与死区（目标在画面中心小范围不动）
  - Q8.8 定点比例控制（KP 增益整定，回路增益 $< 1$ 才不振荡）
  - 舵机 PWM 生成与单帧限速
  - 振荡事故复盘：Kp=90 为什么跑飞 109 度
- **Step4 OSD 与整链**：把结果画回画面（文字/框/延迟可视化）
- 对应文档：`03_vision/step3_closed_loop/`、`step4_osd_integration/`
- 对应金标准：`tools/golden_model.py::gimbal_ctrl_step`